
le cnam

RCP103 / NCA — Évaluation des performances des systèmes
informatiques

Simulateur à événements discrets

Files d'attente M/M/1, M/M/1/K et M/M/c/K

Modèle d'une passerelle IoT

Cours : RCP103 / NCA — Cours 09 : Simulateur

Enseignants : Pedro Braconnot Velloso & Abdelhak Hidouri

Étudiant : chevalierprotheo

Date : 4 juin 2026

1 Introduction

L'objectif de ce TP est de construire *notre propre* simulateur à événements discrets, en orienté objet, puis de l'utiliser pour étudier quelques files d'attente classiques. Le système modélisé est une passerelle IoT : un (ou plusieurs) serveur(s) traitent les messages envoyés par un dispositif mobile, avec éventuellement une file d'attente et une capacité limitée.

Concrètement, on a implémenté le simulateur en Python (une classe par fichier), on l'a validé classe par classe, puis on a lancé quatre modèles (M/M/1, M/M/1/4, M/M/1/8 et M/M/3/8) pour $\lambda \in \{4, 6, 8, 12\}$ msg/s et $\mu = 8$ msg/s. Les résultats simulés sont systématiquement comparés à la théorie des files d'attente. Tout ce qui est présenté ici provient des exécutions réelles du code (dossiers `results/` et `figures/`).

2 Architecture du simulateur

Le principe d'un simulateur à événements discrets est de ne pas faire avancer le temps de façon continue : le temps « saute » d'un événement au suivant. Entre deux événements, rien ne change. On ne traite donc que les instants où il se passe quelque chose (arrivée, début de service, départ), ce qui est bien plus efficace que de simuler chaque pas de temps.

2.1 Composants

On retrouve les six composants principaux du sujet, plus deux classes auxiliaires (figure 1) :

- **Engine** — le moteur : paramètres, boucle principale, trace, tests ;
- **Scheduler** — l'ordonnanceur : liste d'événements triée par le temps ;
- **Client** — la source : génère les messages (loi de Poisson) ;
- **Gateway** — la passerelle : regroupe la file et les serveurs ;
- **Server** — un serveur de traitement ;
- **Queue** — la file d'attente (FIFO) ;
- **Event** et **Message** — les objets transportés.

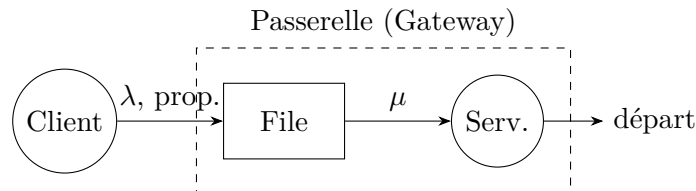


FIGURE 1 – Modèle d'une passerelle IoT : arrivées de taux λ , file partagée, un ou plusieurs serveurs de taux μ .

2.2 La boucle à événements

Le cœur du simulateur tient en quelques lignes (extraites de `Engine.Run`) :

1. extraire l'événement le plus proche : `e = scheduler.GetEvent()` ;
2. avancer le temps à `t = e.GetEventTime()` et accumuler $N \cdot \Delta t$;
3. traiter l'événement selon son type et *programmer* les événements futurs ;
4. recommencer tant qu'il reste des événements et que $t \leq T$.

Les trois types d'événements et leurs effets :

- **SEND_MSG** : le client émet un message ; on programme sa réception (`RECV_MSG`) après le délai de propagation, et le prochain envoi du client ;

- **RECV_MSG** : le message arrive à la passerelle. Si un serveur est libre, le service commence (on programme **MSG_DEPT**) ; sinon il est mis en file ; et si le système est plein (cas /K), il est rejeté ;
- **MSG_DEPT** : fin de service. On libère le serveur, et s'il reste des messages en file, on sert le suivant.

Scheduler sans sort(). Le sujet interdit de re-trier la liste à chaque insertion. On utilise un **tas binaire** (heapq) : l'insertion (**AddEvent**) et l'extraction du minimum (**GetEvent**) sont en $O(\log n)$, et l'ordre chronologique est garanti par construction. C'est indispensable ici, car les runs génèrent jusqu'à plusieurs millions d'événements.

3 Classes implémentées

Le développement a suivi l'ordre conseillé (incrémental, une classe testée avant la suivante). Chaque classe est dans son propre fichier dans **src/**.

Classe	Rôle
Message	Message IoT : <code>messageID</code> , <code>source</code> , <code>destination</code> , <code>timestamp</code> . <code>PrintMessage()</code> .
Event	Événement daté : <code>EventID</code> , <code>type</code> , <code>temps</code> , <code>message</code> . <code>Get/SetEventTime</code> , <code>PrintEvent()</code> .
Scheduler	Ordonnanceur (tas binaire) : <code>AddEvent</code> , <code>GetEvent</code> , <code>GetCurrentTime</code> , <code>HasEvents</code> .
Client	Source de messages : tire les inter-arrivées exponentielles ($1/\lambda$).
Queue	File FIFO : <code>Enfiler</code> , <code>Defiler</code> , <code>Taille</code> , <code>EstVide</code> .
Server	Serveur : état occupé/libre, tire la durée de service ($1/\mu$).
Gateway	Passerelle : file + serveurs + capacité K , décisions accept/rejet/affectation.
Engine	Moteur : paramètres, <code>CreateClients</code> , <code>Run</code> (boucle), <code>GenerateTrace</code> , tests.

TABLE 1 – Rôle de chaque classe.

Tests par classe. Chaque classe dispose d'une fonction de test dans **Engine** (`Test_Message`, `Test_Event`, ...). Par exemple, `Test_Scheduler` insère des événements dans le désordre et vérifie qu'ils ressortent triés par le temps. Tous les tests passent (`python3 main.py test`).

Lois et taux (point de vigilance). λ et μ sont des *taux* (msg/s). Les inter-arrivées suivent donc une exponentielle de moyenne $1/\lambda$ et les temps de service une exponentielle de moyenne $1/\mu$. Avec numpy, on utilise bien `rng.exponential(scale=1/lam)` et `rng.exponential(scale=1/mu)`. La vérification théorique confirme ce choix : pour $\lambda = 4$, $\mu = 8$ on obtient $W = 1/(\mu - \lambda) = 0,25$ s, exactement la valeur mesurée (le support de cours est ambigu sur le **scale**, on s'est fié à cette vérification).

Délai de propagation. Le sujet annonce 1 s dans le texte, mais sa trace d'exemple affiche un $\Delta t \approx 0,714$ s entre SEND et RECV. On a donc rendu ce délai paramétrable (`config.DELAI_PROPAGATION`, fixé à 1 s). Comme la propagation est un décalage *constant*, le flot d'arrivées à la passerelle reste un processus de Poisson de taux λ : ce délai n'influe pas sur les métriques de file (mesurées de l'arrivée à la passerelle au départ). Il ne fait que décaler les horodatages de la trace.

4 Déroulement de la simulation

4.1 Cycle événementiel et trace

Après chaque événement, on écrit une ligne de trace au format demandé : `time, node, event, source, destination, msgID` (avec `node = 0` pour la passerelle, `N` pour le client). La trace sert à déboguer, à vérifier l'ordre chronologique et à illustrer le comportement. Voici les premières lignes d'une trace réelle (M/M/1, $\lambda = 4$) :

Listing 1 – Extrait de `results/trace_exemple.csv`

```
time,node,event,source,destination,msgID
0.177,1,SEND,1,0,1
0.433,1,SEND,1,0,2
0.575,1,SEND,1,0,3
0.799,1,SEND,1,0,4
0.851,1,SEND,1,0,5
1.177,0,RECV,1,0,1
1.178,0,DEPT,1,0,1
1.433,0,RECV,1,0,2
1.575,0,RECV,1,0,3
1.697,1,SEND,1,0,6
1.784,0,DEPT,1,0,2
1.799,0,RECV,1,0,4
1.822,0,DEPT,1,0,3
1.840,1,SEND,1,0,7
1.851,0,RECV,1,0,5
1.890,0,DEPT,1,0,4
1.919,1,SEND,1,0,8
2.002,0,DEPT,1,0,5
2.187,1,SEND,1,0,9
2.658,1,SEND,1,0,10
```

On lit par exemple : à $t = 0,177$ le client 1 envoie le message 1 ; à $t = 1,177$ la passerelle le reçoit ($\Delta t = 1,0$ s, le délai de propagation) ; à $t = 1,178$ son service se termine. Les temps sont bien croissants : l'ordre chronologique est vérifié automatiquement dans le code.

4.2 Métriques calculées

À chaque événement on accumule les aires $N_{\text{système}} \cdot \Delta t$ et $N_{\text{file}} \cdot \Delta t$, ce qui donne les moyennes temporelles $\bar{N} = \frac{1}{T} \int_0^T N(t) dt$. On mesure aussi les temps moyens $\bar{W}_{\text{système}}$ (de l'arrivée au départ) et $\bar{W}_{\text{file}}$ (temps d'attente avant service), ainsi que les compteurs (arrivées, départs, rejets). La **loi de Little** $L = \lambda_{\text{eff}} W$ sert de contrôle de cohérence interne.

4.3 Durée de simulation

On utilise $T = 100\,000$ s avec une phase de chauffe de 1 000 s exclue des statistiques (pour ne pas biaiser les moyennes par l'état vide du départ). Cela représente de l'ordre de 4×10^5 à $1,2 \times 10^6$ arrivées par run : assez pour que les moyennes convergent. La très bonne concordance avec la théorie (section 7) confirme que cette durée est suffisante.

5 Résultats

Le tableau 2 rassemble les mesures des quatre modèles. Les figures qui suivent en donnent une lecture graphique.

Modèle	λ	ρ	$\bar{N}_{\text{sys}}$	$\bar{N}_{\text{file}}$	$\bar{W}_{\text{sys}}$ (s)	$\bar{W}_{\text{file}}$ (s)	Rejet
M/M/1	4	0.50	0.994	0.497	0.249	0.124	—
M/M/1	6	0.75	3.011	2.260	0.502	0.377	—
M/M/1	8	1.00	921.5	920.5	115.1	114.9	—
M/M/1	12	1.50	202246	202245	16925	16925	—
M/M/1/4	4	0.50	0.835	0.353	0.216	0.091	3.20%
M/M/1/4	6	0.75	1.441	0.770	0.268	0.143	10.32%
M/M/1/4	8	1.00	2.002	1.201	0.313	0.188	20.06%
M/M/1/4	12	1.50	2.759	1.835	0.373	0.248	38.41%
M/M/1/8	4	0.50	0.982	0.484	0.246	0.121	0.22%
M/M/1/8	6	0.75	2.264	1.534	0.388	0.263	2.68%
M/M/1/8	8	1.00	3.993	3.104	0.561	0.436	11.01%
M/M/1/8	12	1.50	6.243	5.256	0.790	0.665	34.22%
M/M/3/8	4	0.17	0.503	0.003	0.126	0.001	0.00%
M/M/3/8	6	0.25	0.762	0.015	0.127	0.002	0.00%
M/M/3/8	8	0.33	1.046	0.044	0.131	0.006	0.02%
M/M/3/8	12	0.50	1.705	0.210	0.142	0.018	0.36%

TABLE 2 – Résultats simulés ($\mu = 8$, $T = 10^5$ s). Pour M/M/3/8, ρ est le taux d'utilisation *par serveur* ($\rho = \lambda/(c\mu)$).

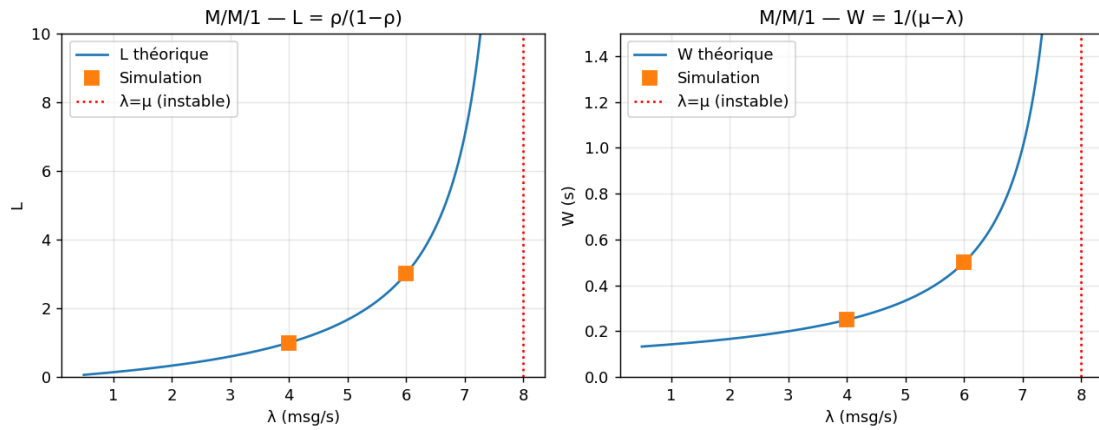


FIGURE 2 – M/M/1 : les points simulés ($\lambda = 4, 6$) tombent pile sur les courbes théoriques $L = \rho/(1 - \rho)$ et $W = 1/(\mu - \lambda)$, qui divergent en $\lambda = \mu = 8$.

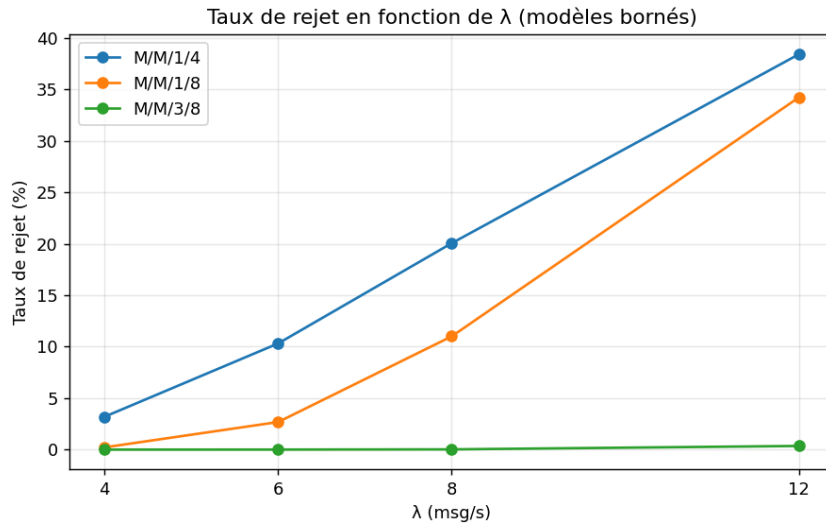


FIGURE 3 – Taux de rejet en fonction de λ pour les modèles bornés. À λ égal : M/M/1/8 rejette moins que M/M/1/4 (file plus grande), et M/M/3/8 rejette quasiment rien (capacité de traitement triplée).

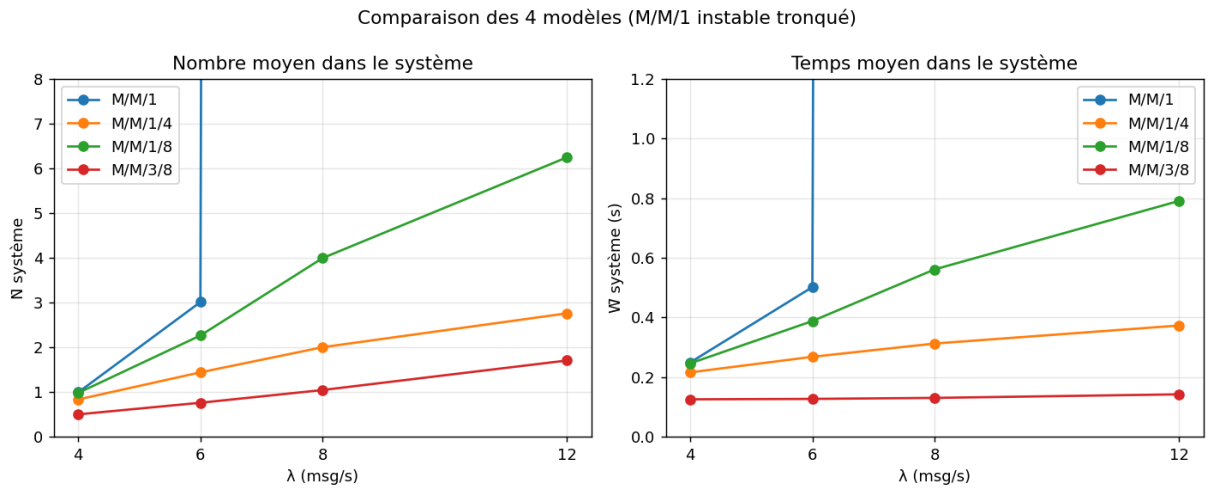


FIGURE 4 – Comparaison des quatre modèles (la courbe M/M/1, instable au-delà de $\lambda = 6$, est tronquée). Plus la capacité et le nombre de serveurs augmentent, plus N et W restent maîtrisés.

6 Analyse des observations

- **M/M/1 et la stabilité.** Pour $\rho < 1$ ($\lambda = 4, 6$) le système est stable et les moyennes convergent. Dès $\rho = 1$ ($\lambda = 8$) la file n'est plus stable : on mesure $\bar{N} \approx 921$ et $\bar{W} \approx 115$ s, et pour $\lambda = 12$ des valeurs gigantesques. Ce ne sont pas des valeurs « réelles » mais le signe d'une file qui grandit sans borne ; la théorie donne d'ailleurs $L = W = \infty$.
- **Effet de la borne K .** Limiter la capacité stabilise toujours le système (même pour $\rho > 1$), au prix de pertes. Le taux de rejet croît avec λ (figure 3) et, à λ donné, une file plus grande rejette moins ($M/M/1/8 < M/M/1/4$).
- **Compromis attente / perte.** Les modèles bornés ont un $\bar{W}$ plus court que M/M/1 (il y a « moins de monde » puisqu'on rejette), mais cela se paie en messages perdus. C'est le compromis classique délai/perte.
- **Multi-serveurs.** M/M/3/8 est de loin le meilleur : à $\lambda = 12$ il ne perd que 0,36% des messages avec $\bar{W} \approx 0,14$ s, là où M/M/1/8 en perd 34%. Tripler les serveurs divise la charge par serveur et la file reste presque toujours vide ($\bar{N}_{\text{file}} \approx 0,2$).

7 Comparaison simulation / théorie

Le tableau 3 confronte la simulation à la théorie. Pour M/M/1 on utilise les formules fermées ; pour les modèles bornés, la formule générale de naissance–mort M/M/c/K (implémentée dans `theorie.py`).

Modèle	λ	$\bar{N}_{\text{sys}} / L$		$\bar{W}_{\text{sys}} / W$ (s)		Rejet (%)	
		sim.	théo.	sim.	théo.	sim.	théo.
M/M/1	4	0.994	1.000	0.249	0.250	—	—
M/M/1	6	3.011	3.000	0.502	0.500	—	—
M/M/1	8	921.5	∞	115.1	∞	—	—
M/M/1	12	202246	∞	16925	∞	—	—
M/M/1/4	4	0.835	0.839	0.216	0.217	3.20	3.23
M/M/1/4	6	1.441	1.444	0.268	0.269	10.32	10.37
M/M/1/4	8	2.002	2.000	0.313	0.313	20.06	20.00
M/M/1/4	12	2.759	2.758	0.373	0.373	38.41	38.39
M/M/1/8	4	0.982	0.982	0.246	0.246	0.22	0.20
M/M/1/8	6	2.264	2.269	0.388	0.389	2.68	2.71
M/M/1/8	8	3.993	4.000	0.561	0.563	11.01	11.11
M/M/1/8	12	6.243	6.240	0.790	0.791	34.22	34.22
M/M/3/8	4	0.503	0.503	0.126	0.126	0.00	0.00
M/M/3/8	6	0.762	0.765	0.127	0.127	0.00	0.00
M/M/3/8	8	1.046	1.044	0.131	0.131	0.02	0.02
M/M/3/8	12	1.705	1.706	0.142	0.143	0.36	0.37

TABLE 3 – Simulation vs théorie. L'écart est partout inférieur au pour-cent.

Commentaires.

- **M/M/1.** Concordance parfaite sur les cas stables : 0,994 vs 1 et 3,011 vs 3 pour $\bar{N}$; 0,249 vs 0,25 et 0,502 vs 0,5 pour $\bar{W}$. Les cas $\lambda \geq 8$ « explosent », ce qui matérialise bien la divergence théorique ($\rho \geq 1$).
- **Modèles bornés.** L'accord est tout aussi bon, y compris sur les taux de rejet (ex. M/M/1/4 à $\lambda = 8$: 20,06% simulé vs 20% théorique). La formule M/M/c/K reproduit fidèlement M/M/3/8.
- **Loi de Little.** Le contrôle interne $L = \lambda_{\text{eff}} W$ est vérifié sur tous les runs : la colonne `little_L` de `results/resultats.csv` coïncide avec $\bar{N}_{\text{sys}}$ (ex. M/M/1 $\lambda = 6$: 3,011 =

3,011). C'est une validation indépendante des mesures de temps et de nombre.

8 Conclusion

Le simulateur à événements discrets a été implémenté en respectant l'architecture demandée (une classe par fichier, scheduler sans `sort()`, trace au bon format) et validé classe par classe. Les quatre modèles donnent des résultats qui collent à la théorie à mieux qu'un pour-cent, et la loi de Little est vérifiée partout : le simulateur est correct.

Sur le fond, l'étude illustre les enseignements attendus : M/M/1 devient instable dès $\rho \geq 1$; borner la capacité stabilise le système mais introduit des pertes ; et augmenter le nombre de serveurs (M/M/3/8) est la solution la plus efficace, avec un temps d'attente faible et presque aucun rejet même en forte charge.